

Nifty 100 Financial Intelligence Platform - Analyst Guide

Overview

This platform provides comprehensive financial analysis of 92 Nifty 100 companies through a complete pipeline:

- Data extraction from Excel files
- ETL processing into SQLite database
- KPI calculation engine (27+ ratios)
- Financial screening and peer comparison
- Interactive dashboard and REST API
- Automated report generation (tearsheets, sector reports)

Data Pipeline

1. Data Sources

- Raw Excel files in `~/data/raw/`` (read-only)
- 12 source files covering P&L, balance sheet, cash flow, etc.
- Processed into 12-table SQLite database (`db/nifty100.db``)

2. ETL Process

Run the ETL pipeline:

```
```bash
python -m src.etl.loader
```
```

This loads/refreshes the database with cleaned and validated data.

3. KPI Engine

Calculate all financial ratios:

```
```bash
```

```
python -m src.analytics.ratios
```

```
```
```

Computes 27+ KPIs including:

- Profitability: NPM, OPM, ROE, ROCE
- Leverage: D/E, Interest Coverage
- Efficiency: Asset Turnover
- Cash Flow: FCF, CAGR metrics
- Valuation: P/E ratio

Key Features

Financial Ratios (Sprint 2)

All monetary values in Indian Rupees (Crore) unless specified.

****Core Formulas:****

- $NPM = \text{Net Profit} / \text{Sales} \times 100$
- $OPM = \text{Operating Profit} / \text{Sales} \times 100$
- $ROE = \text{Net Profit} / (\text{Equity Capital} + \text{Reserves}) \times 100$
- $ROCE = (\text{Operating Profit} - \text{Depreciation}) / (\text{Equity Capital} + \text{Reserves} + \text{Borrowings}) \times 100$
- $D/E = \text{Borrowings} / (\text{Equity Capital} + \text{Reserves})$
- $ICR = (\text{Operating Profit} + \text{Other Income}) / \text{Interest}$
- $FCF = \text{Operating Activity} + \text{Investing Activity}$
- $\text{Asset Turnover} = \text{Sales} / \text{Total Assets}$
- $CAGR = ((\text{End}/\text{Start})^{(1/n)} - 1) \times 100$

****Edge Cases Handled:****

- Division by zero → Returns None
- Negative denominator → Returns None
- Missing/null inputs → Returns None
- Debt-free companies → Interest Coverage = None (display "Debt Free")

Financial Screener (Sprint 3)

Predefined screens for different investment strategies:

1. ****Growth Investor****: High revenue growth, strong ROE, moderate debt
2. ****Value Investor****: Low P/E, high dividend yield, strong fundamentals
3. ****Dividend Yield****: Consistent dividend payers
4. ****Quality at Reasonable Price****: Strong ROE, low debt, reasonable valuation
5. ****Turnaround Opportunity****: Recovering companies with improving fundamentals
6. ****Financial Stability****: Low debt, strong cash flow, consistent profitability

Custom screens available via POST `/api/v1/screener/custom` with JSON filters.

REST API (Sprint 6)

FastAPI-based REST interface with comprehensive endpoints:

****Companies:****

- GET `/api/v1/companies/` - List all companies
- GET `/api/v1/companies/{ticker}` - Company profile
- GET `/api/v1/companies/{ticker}/pl` - P&L; statement
- GET `/api/v1/companies/{ticker}/bs` - Balance sheet
- GET `/api/v1/companies/{ticker}/cashflow` - Cash flow statement
- GET `/api/v1/companies/{ticker}/ratios` - Financial ratios

****Screener:****

- GET `/api/v1/screener/` - Custom screening with query params

- GET `/api/v1/screener/presets` - Get predefined screens
- GET `/api/v1/screener/run/{preset\_id}` - Run a preset
- POST `/api/v1/screener/custom` - Custom screening with JSON

****Sectors:****

- GET `/api/v1/sectors/` - Sector summary with medians
- GET `/api/v1/sectors/{sector}/companies` - Companies in sector

****Health:****

- GET `/api/v1/health` - System health check

Dashboard (Sprint 4)

Streamlit-based interactive dashboard:

```
```bash
```

```
streamlit run src/dashboard/app.py
```

```
```
```

Features 8 screens including company analysis, sector comparison, screening tools, and visualization modules.

Reports Generation (Sprint 5)

Automated report creation:

- Company tearsheets (PDF, 2 pages each)
- Sector reports (PDF)
- Portfolio summary (PDF)
- Pros/Cons analysis (CSV)
- Cash flow intelligence (Excel)

Run report generation:

```
```bash
```

## Tearsheets

```
python -m src.reports.tearsheet
```

## Sector reports

```
python -m src.reports.sector_report
```

## Portfolio summary

```
python -m src.reports.portfolio_report
```

## Pros/Cons generation

```
python -m src.nlp.pros_cons_generator
```

## Cash flow intelligence

```
python -m src.analytics.cashflow_kpis
```

```
...
```

## Testing Strategy

Comprehensive test suite ensures reliability:

- Unit tests for all modules
- Integration tests for API endpoints
- Data quality validation tests
- 356 tests passing as of last verification

Run all tests:

```
```bash
```

```
pytest tests/ -v
```

```
...
```

Deployment

The platform consists of:

- **Backend**: FastAPI server (port 8000)
- **Frontend**: Streamlit dashboard (port 8501)
- **Database**: SQLite (file-based)

Start both services:

```
```bash
```

## Terminal 1: API server

```
uvicorn src.api.main:app --port 8000
```

## Terminal 2: Dashboard

```
streamlit run src/dashboard/app.py
```

```
```
```

Data Quality

All source Excel files are treated as read-only. The platform implements 16 data quality rules (DQ-01 through DQ-16) covering:

- Primary key uniqueness
- Foreign key integrity
- Balance sheet validation
- Data completeness
- Consistency checks

Technical Stack

- **Language**: Python 3.14+
- **Framework**: FastAPI (API), Streamlit (Dashboard)
- **Database**: SQLite
- **Data Processing**: Pandas

- **Testing**: Pytest
- **PDF Generation**: ReportLab
- **Machine Learning**: Scikit-learn (for clustering)

Project Structure

...

data/raw/ Source Excel files (read-only)

db/

nifty100.db SQLite database

src/

etl/ Extract, Transform, Load

analytics/ KPI engines, ratios, CAGR, cash flow

screeners/ Screening logic

dashboard/ Streamlit application

reports/ PDF and report generation

nlp/ Natural language processing for pros/cons

api/ FastAPI endpoints

tests/ Test suite

output/ Generated CSV/Excel reports

logs/ Runtime logs

reports/ Generated PDF reports

...

Getting Started

1. Ensure virtual environment is activated (`venv\Scripts\activate`)
2. Install dependencies: `pip install -r requirements.txt`
3. Load initial data: `python -m src.etl.loader`

4. Calculate KPIs: ``python -m src.analytics.ratios``
5. Start API: ``uvicorn src.api.main:app --port 8000``
6. Start dashboard: ``streamlit run src/dashboard/app.py``
7. Access dashboard at <http://localhost:8501>
8. Access API docs at <http://localhost:8000/docs>

Support

For issues or questions, refer to:

- `SPRINT1_RETROSPECTIVE.md` for initial lessons learned
- Individual module documentation in source code
- Test files for usage examples